

Experiment No.:6

Threat Modeling and Security Hardening of an IoE Smart Banking System Using Python (Google Collab)

Aim

To develop and simulate a Threat Modeling and Security Hardening system for an IoE Smart Banking environment using Python in Google Colab that securely authenticates users, simulates ATM transactions, detects potential cyber threats, classifies transactions as SUCCESS or BLOCKED, and visualizes security statistics using graphs.

Objective

The objective of this experiment is to simulate a secure IoE Smart Banking System using Python in Google Colab. The system authenticates a user using SHA-256 password hashing and simulates ATM transactions from multiple ATMs. Each transaction is analyzed for possible cyber threats such as ATM Spoofing, Brute Force attacks, Data Tampering, and DoS attacks. Based on the detected threat, the transaction is classified as either SUCCESS or BLOCKED. The system generates a transaction report, displays a security summary, and visualizes transaction amounts, transaction status, threat distribution, and customer transaction amounts using graphical representations. This experiment demonstrates the concepts of threat modeling, security hardening, authentication, attack detection, and real-time security analysis in IoE banking systems.

Software Requirements

- Google Colab
- Python 3.x

Python Libraries Used

- pandas
- matplotlib
- hashlib
- random

```

# =====

# Threat Modeling & Security Hardening

# IoE Smart Banking System

# Google Colab

# =====


import random

import hashlib

import pandas as pd

import matplotlib.pyplot as plt


print("="*60)

print(" IoE SMART BANKING SECURITY SYSTEM ")

print("="*60)


# -----

# User Authentication

# -----


username = "admin"

password = "Bank@123"


password_hash = hashlib.sha256(password.encode()).hexdigest()


print("\nLogin Successful")

print("Username :", username)

```

```

print("Password Hash :", password_hash[:20], "...")

# -----
# Smart ATM Transactions
# -----

atm_ids = ["ATM001", "ATM002", "ATM003", "ATM004", "ATM005"]

records = []

for atm in atm_ids:

    customer = random.randint(1001, 1050)
    amount = random.randint(500, 10000)

    threat = random.choice([
        "SAFE",
        "ATM Spoofing",
        "Brute Force",
        "Data Tampering",
        "DoS Attack"
    ])

    if threat == "SAFE":
        status = "SUCCESS"
    else:

```

```
status="BLOCKED"
```

```
records.append([
```

```
    atm,
```

```
    customer,
```

```
    amount,
```

```
    threat,
```

```
    status
```

```
])
```

```
# -----
```

```
# Display Transactions
```

```
# -----
```

```
df = pd.DataFrame(records,columns=[
```

```
    "ATM ID",
```

```
    "Customer ID",
```

```
    "Amount",
```

```
    "Threat",
```

```
    "Status"
```

```
])
```

```
print("\nATM Transaction Report\n")
```

```
print(df)
```

```
# -----
```

```
# Security Summary
```

```
# -----
```

```
safe = len(df[df["Threat"]=="SAFE"])
```

```
spooof = len(df[df["Threat"]=="ATM Spoofing"])
```

```
brute = len(df[df["Threat"]=="Brute Force"])
```

```
tamper = len(df[df["Threat"]=="Data Tampering"])
```

```
dos = len(df[df["Threat"]=="DoS Attack"])
```

```
print("\n===== SECURITY REPORT =====")
```

```
print("Safe Transactions :",safe)
```

```
print("ATM Spoofing      :",spooof)
```

```
print("Brute Force       :",brute)
```

```
print("Data Tampering    :",tamper)
```

```
print("DoS Attack        :",dos)
```

```
# -----
```

```
# GRAPH 1
```

```
# Transaction Amount
```

```
# -----
```

```
plt.figure(figsize=(8,4))
```

```
plt.bar(df["ATM ID"],df["Amount"])
```

```
plt.title("ATM Transaction Amount")
```

```
plt.xlabel("ATM")
```

```
plt.ylabel("Amount")
```

```
plt.grid(axis="y")
```

```
plt.show()
```

```
# -----
```

```
# GRAPH 2
```

```
# Success vs Blocked
```

```
# -----
```

```
success = len(df[df["Status"]=="SUCCESS"])
```

```
blocked = len(df[df["Status"]=="BLOCKED"])
```

```
plt.figure(figsize=(5,4))
```

```
plt.bar(["SUCCESS","BLOCKED"],[success,blocked])
```

```
plt.title("Transaction Status")
```

```
plt.ylabel("Count")
```

```
plt.show()
```

```
# -----
```

```
# GRAPH 3
```

```
# Threat Distribution
```

```
# -----
```

```
plt.figure(figsize=(6,6))
```

```
plt.pie(
```

```
    [safe,spoof,brute,tamper,dos],
```

```
    labels=[
```

```

        "SAFE",
        "ATM Spoofing",
        "Brute Force",
        "Tampering",
        "DoS"
    ],
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Threat Distribution")
plt.show()

# -----
# GRAPH 4
# Customer Transaction Amount
# -----

plt.figure(figsize=(8,4))
plt.plot(df["Customer ID"],df["Amount"],marker="o")
plt.title("Customer Transaction Amount")
plt.xlabel("Customer ID")
plt.ylabel("Amount")
plt.grid(True)
plt.show()

print("\n=====")

```

```
print("Threat Modeling Completed Successfully")  
print("Security Hardening Applied")  
print("System Protected Successfully")  
print("=====")
```

Theory

An IoE (Internet of Everything) Smart Banking System connects ATMs, banking servers, customers, and security systems through intelligent communication networks. Since banking systems handle sensitive financial information, they are common targets for cyber attacks such as ATM Spoofing, Brute Force attacks, Data Tampering, and Denial-of-Service (DoS) attacks.

Threat Modeling is the process of identifying possible security threats before they cause damage. Security Hardening strengthens the system by implementing protective mechanisms such as secure authentication, password hashing, transaction monitoring, and attack detection.

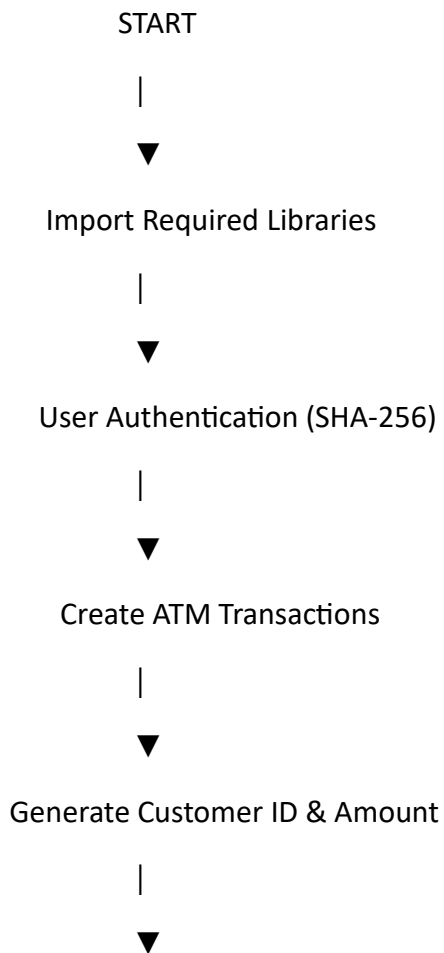
In this experiment, the user password is protected using the SHA-256 hashing algorithm. Random ATM transactions are generated and each transaction is checked for security threats. Transactions without threats are marked as SUCCESS, while suspicious transactions are BLOCKED. The system also generates graphical reports to help visualize security information.

Algorithm

1. Start the program.
2. Import all required Python libraries.
3. Create the username and password.
4. Encrypt the password using SHA-256 hashing.
5. Display successful login information.
6. Create five ATM IDs.
7. Generate random customer IDs and transaction amounts.
8. Randomly generate security threats.
9. Classify each transaction as SUCCESS or BLOCKED.

10. Store all transaction details.
11. Display the ATM transaction report.
12. Generate the security summary.
13. Display the transaction amount graph.
14. Display the Success vs Blocked graph.
15. Display the Threat Distribution pie chart.
16. Display the Customer Transaction Amount graph.
17. Display "Threat Modeling Completed Successfully".
18. Stop the program.

Flowchart



Randomly Generate Threat Type



Is Threat = SAFE ?



YES



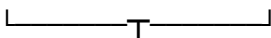
NO



SUCCESS



BLOCKED



Store Transaction Details



Display Transaction Report



Generate Security Summary



Display Security Graphs



Display Final Status





END

Threat Types Used

Threat	Transaction Status
SAFE	SUCCESS
ATM Spoofing	BLOCKED
Brute Force	BLOCKED
Data Tampering	BLOCKED
DoS Attack	BLOCKED

Program Description

The Python program simulates an IoE Smart Banking Security System. Initially, the user is authenticated using a SHA-256 encrypted password. The system then generates ATM transactions for five different ATMs with random customer IDs and transaction amounts. Each transaction is assigned a randomly generated threat type. If no threat is detected, the transaction is marked as SUCCESS; otherwise, it is marked as BLOCKED. The generated transaction data are stored in a pandas DataFrame and displayed as a transaction report. A security summary is then generated, followed by graphical visualizations showing transaction amounts, transaction status, threat distribution, and customer transaction amounts. Finally, the program displays a successful completion message.

Input

- Username = admin
- Password = Bank@123
- ATM IDs = ATM001 to ATM005
- Random Customer IDs
- Random Transaction Amounts

- Random Threat Types

Output

- User Authentication Result
- ATM Transaction Report
- Security Summary
- Transaction Amount Bar Graph
- Success vs Blocked Graph
- Threat Distribution Pie Chart
- Customer Transaction Amount Line Graph
- Final Security Status

Sample Output

ATM ID	Customer ID	Amount	Threat	Status
ATM001	1008	3500	SAFE	SUCCESS
ATM002	1032	7800	Brute Force	BLOCKED
ATM003	1015	5200	SAFE	SUCCESS
ATM004	1044	9400	ATM Spoofing	BLOCKED
ATM005	1006	2600	SAFE	SUCCESS

Observations

- User authentication is completed using SHA-256 hashing.
- ATM transactions are generated automatically.
- Threats are randomly assigned to transactions.
- Unsafe transactions are blocked.

- A complete security report is generated.
- Graphs provide visual analysis of banking security.
- The system successfully simulates threat detection in IoE banking.

Applications

- Smart Banking Systems
- ATM Security Monitoring
- Online Banking Security
- Cybersecurity Training
- IoT and IoE Banking Research
- Fraud Detection Systems
- Financial Network Security
- Banking Security Analytics

Advantages

- Secure password hashing using SHA-256.
- Simulates real-world banking security.
- Automatic threat detection.
- Easy to understand.
- Provides graphical analysis.
- Useful for educational purposes.
- Can be extended with real banking databases.
- Demonstrates cybersecurity concepts effectively.

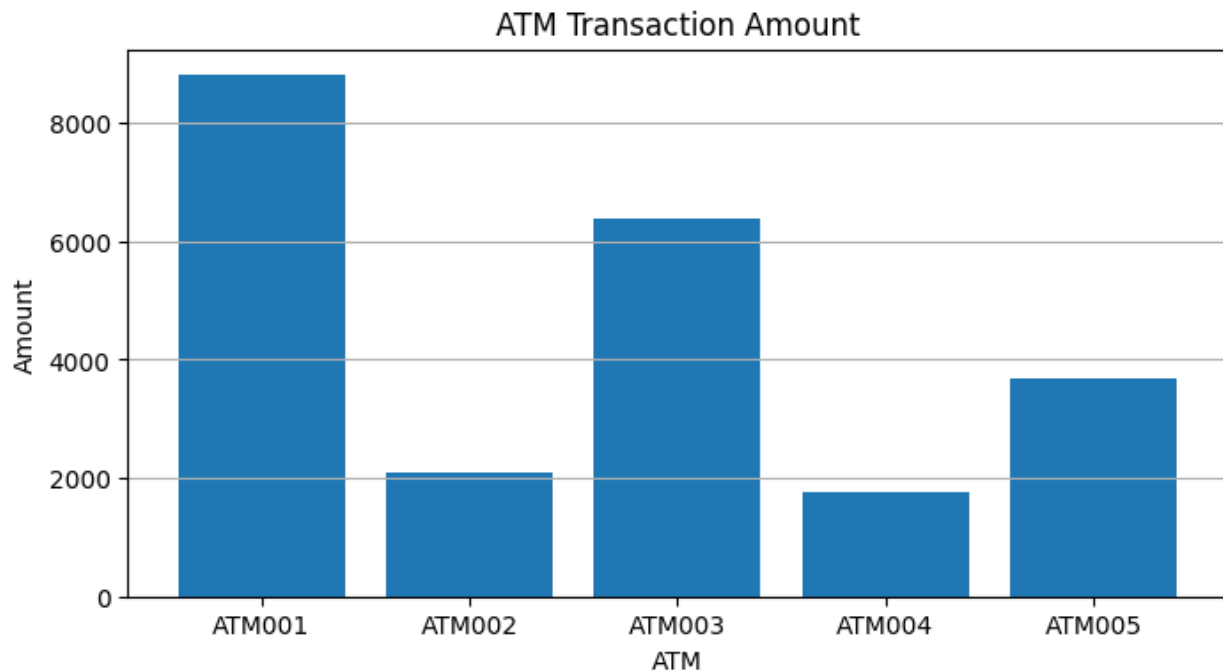
Limitations

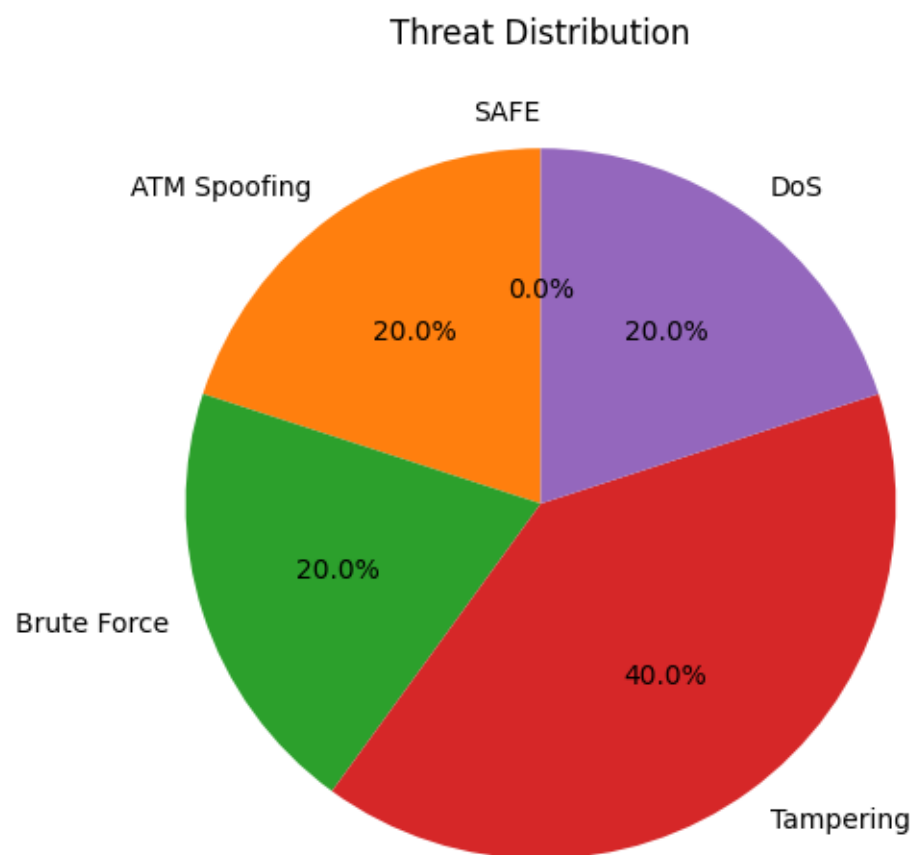
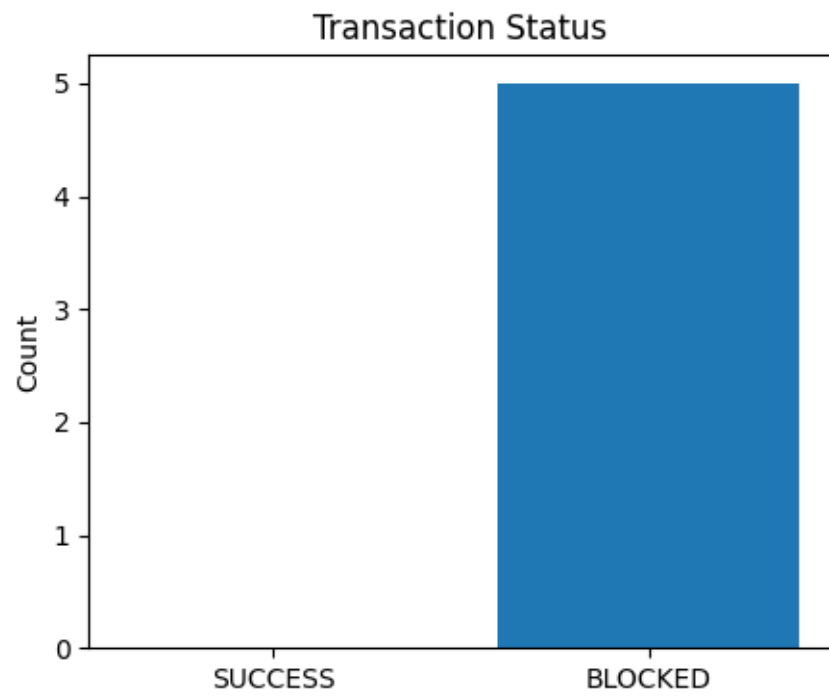
- Uses simulated transaction data.

- No real banking server connection.
- Threats are randomly generated.
- Does not send real-time notifications.
- Suitable only for educational demonstrations.

Result

The **Threat Modeling and Security Hardening of the IoE Smart Banking System** was successfully simulated using Python in Google Colab. The system authenticated users using SHA-256 password hashing, generated ATM transactions, detected cyber threats such as ATM Spoofing, Brute Force attacks, Data Tampering, and DoS attacks, and classified transactions as **SUCCESS** or **BLOCKED**. Transaction reports, security summaries, and graphical visualizations were generated successfully. Thus, the experiment successfully demonstrated authentication, threat modeling, security hardening, transaction monitoring, attack detection, and security visualization in a simulated IoE Smart Banking environment.





Threat Modeling Completed Successfully

Security Hardening Applied

System Protected Successfully

